

Verkapselung von Tk-Objekten für HTk

ludi@tzi.de

Inhaltsverzeichnis

1	Grundlegendes	1
2	Externe Repräsentation der Objekte und Konstruktion	1
2.1	Typ	1
2.2	Container	1
2.3	Konstruktor	2
3	Der Typ GUIOBJECT und die Klasse GUIObject	2
4	Konfigurationsoptionen	3
4.1	Der Typ GUIVALUE und die Klasse GUIValue	3
4.2	Interne Repräsentation	4
4.3	Externe Repräsentation	4
4.4	cset und cget	4
4.5	Vordefinierte Klassen	5
4.6	Objekt-spezifische Konfigurationen	5
5	Ein internes GUIOBJECT erzeugen	5
5.1	Widgets	5
5.2	Canvasitems	7
5.3	Menüelemente	8
6	Objekt-Methoden	8
6.1	Tcl-Syntax	8
6.2	Tcl-Skripte und Kommandos ausführen	8
6.3	Der Methods-Typ	9

1 Grundlegendes

Im folgenden gebe ich einen Überblick über die Verkapselung von Tk-Objekten für HTk. Dies ist keine komplette Einführung in die Funktionsweise von HTk, insbesondere wird man Informationen über die Funktion des HTk-Kernels vermissen. Dennoch sollte es danach leichter sein, sich weiter in die HTk-Interna einzuarbeiten, denn die wesentlichen Typen und einige wichtige Funktionen werden hier erläutert. Vorausgesetzt werden die grundlegende Kenntnis der Tcl/Tk-Syntax (z.B. zur Objekterzeugung oder deren Konfiguration) und natürlich Haskell-Kenntnisse.

Außerdem wäre es gut, schon ein bißchen *in* HTk programmiert oder sich zumindest einige der Beispiele angesehen zu haben.

2 Externe Repräsentation der Objekte und Konstruktion

2.1 Typ

Zentral bei der Verkapselung eines Tk-Objektes für HTk ist der (abstrakt) exportierte Datentyp. Dieser ist die externe Repräsentation des jeweiligen Objektes und dient später als Handle, u.a. um nachträglich Konfigurationsoptionen zu setzen oder Events an ein Objekt zu binden. Für das Label-Widget sieht dieser beispielsweise so aus:

```
newtype Label a = Label GUIOBJECT deriving Eq
-- ein (Label String) enthält Text, ein (Label Image) ein Bild
```

Dieser Typ kann weitere Objekt-spezifische Felder haben, das ist eine Frage der individuellen Implementierung. Er enthält jedoch immer ein Feld des Typs `GUIOBJECT` (siehe übernächster Abschnitt).

2.2 Container

Objekte, die andere Objekte enthalten, die dann mittels `pack` (d.h. durch den *pack geometry manager*) oder `grid` (d.h. durch den *grid geometry manager*) gepackt werden, müssen Instanzen der Klasse `Container` sein. Diese Klasse enthält keine Methoden, eine Instanz dieser Klasse wird jedoch immer vorausgesetzt, um ein solches Objekt als Vaterobjekt bei der Erzeugung anderer Objekte angeben zu können. Instanzen dieser Klasse sind beispielsweise Fenster, Frames, aber auch Canvases (denn diese können wieder Widgets in Embedded Windows enthalten).

2.3 Konstruktor

Desweiteren wird immer ein Konstruktor exportiert, der zur Erzeugung der Objekte dient. Hier unterscheiden wir zwei Schreibweisen:

- Für Objekte, die nach der Erzeugung noch gepackt werden müssen (das sind eigentlich nur Widgets), beginnt der Konstruktor mit `new`, z.B. für ein Label:

```
newLabel :: Container par => par ->           -- Vaterobjekt
                        [Config (Label a)] -> -- Konfigurationen
                        IO (Label a)
```

- Für Objekte, die nicht explizit gepackt werden müssen (z.B. Canvasitems oder Menüelemente) beginnt der Konstruktor mit `create`, z.B. für ein Rechteck auf einem Canvas:

```
createRectangle :: Canvas ->           -- enthaltender Canvas
                [Config Rectangle] -> -- Konfigurationen
                IO Rectangle
```

Bei der Erzeugung wird normalerweise ein Vaterobjekt angegeben (außer z.B. bei Fenstern). Dieses muß für Widgets, die später mittels `pack` oder `grid` gepackt werden, eine Instanz der Klasse `Container` sein (s.o.). Weiter wird meist eine Liste von Objekt-Konfigurationen angegeben, um deren Verarbeitung man sich im Konstruktor kümmern muß. Ein Beispiel für einen Konstruktor gibt es später, zunächst jedoch etwas mehr zur internen Repräsentation der Objekte und deren Erzeugung und Konfiguration.

3 Der Typ `GUIOBJECT` und die Klasse `GUIObject`

Der Typ `GUIOBJECT` ist die interne Repräsentation der verschiedenen Oberflächen-Objekte (im wesentlichen Widgets, Canvas-Objekte, Menüelemente, aber auch spezielleres wie z.B. Text-Tags oder Canvas-Tags). Diese interne Repräsentation durch *einen* Typen ermöglicht z.B. Listen verschiedener Objekte im internen Status zu halten oder als Funktions-Argumente zu übergeben.

Der `GUIOBJECT`-Wert muß jedoch normalerweise nicht selbst konstruiert werden, hierzu gibt es spezielle Funktionen (z.B. `createGUIObject` im Modul `Core`, `createCanvasItem` im Modul `CanvasItemAux`). Aussehen tut er wie folgt:

```
data GUIOBJECT = GUIOBJECT ObjectID (Ref OST) | ROOT
```

Den `ROOT`-Konstruktor möchte ich hier nicht weiter erläutern. Er hat eine spezielle Bedeutung und steht für die Wish-Instanz bzw. das Hauptfenster, dies muß man aber jetzt nicht verstehen. Für „normale“ GUI-Objekte wird ein dem ersten Konstruktor entsprechender Wert erzeugt. Dieser enthält die `ObjectID`, die dieses spezielle Objekt eindeutig identifiziert und eine Referenz auf das zu diesem Objekt gehörende Statusobjekt (die `ObjectID` und das Statusobjekt werden jedoch von obiger Funktion automatisch erzeugt, daher will ich hier auch darauf nicht weiter eingehen).

Der externe Typ muß immer eine Instanz der Klasse `GUIObject` sein:

```
class GUIObject w where
  toGUIObject      :: w -> GUIOBJECT  -- Wandeln in interne Repräsentation
  cname            :: w -> String      -- textuelle Repräsentation
  cset              :: GUIValue a => w -> ConfigID -> a -> IO w    -- vordefiniert
  cget              :: GUIValue a => w -> ConfigID -> IO a         -- vordefiniert
```

Die Instanz für ein Objekt sieht dann z.B. für ein Label aus wie folgt:

```
instance GUIObject (Label a) where
  toGUIObject (Label w) = w
  cname _ = "Label"
```

So kann bei Bedarf durch die Klassenmethode `toGUIObject` der externe Typ in die interne Repräsentation gewandelt werden. Die `cname`-Methode liefert lediglich eine textuelle Repräsentation des Objekttyps und kann z.B. für Debug-Ausgaben benutzt werden. Die Methoden `cset` und `cget` sind wichtig bei der Implementierung von Objekt-Konfigurationen und werden im folgenden besprochen.

4 Konfigurationsoptionen

4.1 Der Typ GUIVALUE und die Klasse GUIValue

Eine ähnliche Verallgemeinerung wie beim Umgangs mit Objekten gibt es auch für Werte, die die verschiedenen Konfigurationsoptionen der Tk-Objekte annehmen können. Hierzu gibt es zunächst wieder *einen* Typen, durch den alle Werte dieser Art intern dargestellt werden. Dies ist der Typ `GUIVALUE`:

```
data Generator = HaskellTk | Tk
data GUIVALUE = GUIVALUE Generator String
```

Entsprechend gibt es auch wieder eine Klasse `GUIValue`, die im wesentlichen zum Wandeln in die interne Repräsentation und Angabe von default-Werten dient. Der Typ eines Konfigurationswertes muß also immer eine Instanz der Klasse `GUIValue` sein:

```
class (Show a, Read a) => GUIValue a where
    cdefault      :: a          -- default-Wert
    toGUIValue    :: a -> GUIVALUE -- wandeln in GUIVALUE
                                -- für gängige Fälle vordefiniert
    maybeGUIValue :: GUIVALUE -> (Maybe a) -- intern, vordefiniert
    fromGUIValue  :: GUIVALUE -> a        -- intern, vordefiniert
```

Der Typ `Generator` ist hier eigentlich uninteressant. Er wird nur intern bei der Rückgabe von Tk-Werten, die ja immer als Strings von der Wish gelesen werden, benutzt. Bei der Implementierung der Methode `toGUIValue` in der `GUIVALUE`-Instanz wird man also immer `HaskellTk` als `Generator` angeben. Das zweite Feld des `GUIVALUE`-Konstruktors muß die Tk-Repräsentation des entsprechenden Wertes als String, genau so wie dieser auf die Wish ausgegeben werden soll, enthalten. Diese Repräsentation sollte die `Show`-Instanz, die von der `GUIValue`-Instanz vorausgesetzt wird liefern (falls dies aus irgendwelchen Gründen einmal anders ist, reicht es allerdings, die `toGUIObject`-Methode entsprechend zu implementieren). Umgekehrt *muß* die ebenfalls vorausgesetzte `Read`-Instanz die Tcl-Repräsentation in den zugehörigen HTk-Wert wandeln.

Die `toGUIValue` Methode muß man allerdings in der Regel nicht überschreiben. Sie benutzt sonst einfach die vorausgesetzte `Show`-Instanz und liefert einen entsprechenden `GUIVALUE` mit `HaskellTk` als `Generator`. Zu beachten ist dabei allerdings, daß gewisse Zeichen durch entsprechende Escape-Sequenzen ersetzt werden und die Zeichenkette in Anführungszeichen gesetzt wird, damit keine Probleme mit enthaltenen Leerzeichen auftreten. Normalerweise ist dies so gewollt, kann aber in bestimmten Situationen auch unerwünscht sein, so daß dann die `toGUIValue`-Methode überschrieben werden muß.

Die `cdefault`-Methode liefert schließlich einen default-Wert für diesen Typen. Normalerweise stehen hier die Tcl-Defaults. Soll ein default-Wert abweichend davon gelten, muß dieser bei der Erzeugung mit gesetzt werden (dazu siehe Objekt-Methoden $\rightarrow$ `createCmd`).

4.2 Interne Repräsentation

Die interne Repräsentation einer Objekt-Konfiguration ist ein Wert vom Typ `ConfigOption`:

```
type ConfigID = String
type ConfigOption = (ConfigID, GUIVALUE)
```

Dieser Typ ist ein Tupel aus `ConfigID` und `GUIVALUE`. Die `ConfigID` ist dabei der Tcl-Bezeichner der entsprechenden Konfigurationsoption als `String` (ohne das in Tcl beim Setzen von Konfigurationen übliche Minus, z.B. `font` für eine Schriftart), und der `GUIVALUE` der Wert der Konfiguration (s.o.).

4.3 Externe Repräsentation

Nach außen hin sind Konfigurationen einfach Aktionen, die auf das entsprechende Objekt angewendet werden können. Sie werden somit wieder durch *einen* (Objekt-spezifischen) Typen dargestellt, so daß z.B. eine Liste zu setzender Konfigurationen bei der Erzeugung eines Objektes angegeben werden kann. Dies ist der folgende Typ:

```
type Config w = w -> IO w
```

Somit ist z.B. eine Konfiguration für ein Canvas-Widget vom Typ `Config Canvas`.

4.4 cset und cget

Bei der Implementierung von Funktionen, die Objekt-Konfigurationen erzeugen oder abfragen, können die Methoden `cset` und `cget` der Klasse `GUIObject` benutzt werden:

```
cset :: GUIValue a => w -> ConfigID -> a -> IO w
cget :: GUIValue a => w -> ConfigID -> IO a
```

`cset` dient dabei zum Setzen einer Konfiguration und `cget` zum Abfragen des Wertes einer Konfiguration. Die Benutzung sollte anhand der obigen Ausführungen klar sein.

4.5 Vordefinierte Klassen

Für Konfigurationsoptionen, die für mehrere Objekte benutzt werden, gibt es im Modul `Configuration` vordefinierte Klassen. Dies ermöglicht u.a. eine einheitliche Namensgebung und die mehrfache Verwendung einmal implementierter Methoden. Manchmal ist es aber auch sinnvoll trotz unterschiedlicher Tcl-Syntax die Methoden einer solchen Klasse zu überschreiben, um zumindest auf HTk-Seite eine einheitliche (möglicherweise sinnvollere) Namensgebung zu erreichen.

Ein Beispiel (die Klasse `HasSize`, deren Instanz fast alle Oberflächenobjekte sind):

```
class GUIObject w => HasSize w where
  width      :: Distance -> Config w    -- Breite setzen
  getWidth   :: w -> IO Distance       -- Breite abfragen
  height     :: Distance -> Config w    -- Höhe setzen
  getHeight  :: w -> IO Distance       -- Höhe abfragen
  size       :: Size -> Config w        -- Breite und Höhe setzen
  getSize    :: w -> IO Size           -- Breite und Höhe abfragen
```

```

-- Implementierung der Klassenmethoden
width s w      = cset w "width" s
getWidth w     = cget w "width"
height s w     = cset w "height" s
getHeight w    = cget w "height"
size (x,y) w   = width x w >> height y w
getSize w      = getWidth w >>= \ x -> getHeight w >>= \ y -> return (x,y)

```

4.6 Objekt-spezifische Konfigurationen

Funktionen, die nur einfach verwendete Objekt-Konfigurationen setzen bzw. deren Werte abfragen werden als einzelne Funktionen in den entsprechenden Modulen implementiert. Da sie sich ansonsten nicht von den im vorigen Abschnitt beschriebenen Klassenmethoden unterscheiden, wären Sie hier im Prinzip gar nicht erwähnenswert, wenn nicht *HDoc*, ein javadoc-ähnliches Dokumentationstool das wir verwenden, leider ein Problem mit dem oben beschriebenen `Typ Config w` hätte. Da aber HDoc aber ein Problem mit Funktionstypen hat, gibt es leider mit obiger Schreibweise eine *parse error*. Daher muß in diesem Fall der `Typ Config w` ausgeschrieben werden (also als `w -> IO w`), zumindest bis eine spätere Version hoffentlich damit zurechtkommt.

5 Ein internes GUIOBJECT erzeugen

5.1 Widgets

Der interne GUIOBJECT-Wert wird für Widgets durch die Funktion

```

createGUIObject :: GUIOBJECT ->      -- Vaterobjekt
                  ObjectKind ->      -- Objekttyp
                  Methods ->         -- Objekt-spezifische Zugriffsmethoden
                  IO GUIOBJECT

```

im Modul `Core` erzeugt werden. Hierbei müssen die folgenden Parameter angegeben werden:

- Das Vaterobjekt: Die interne Repräsentation (als `GUIOBJECT`) des Containers, der das Widget enthalten soll. Obwohl Widgets erst später gepackt werden, muß dieser bereits bei der Konstruktion angegeben werden, da der Tcl-Pfad zu dem erzeugten Objekt entsprechend (hierarchisch) gesetzt werden muß.
- Der Objekttyp: Um auch anhand der internen `GUIOBJECT`-Werte verschiedene Objekte unterscheiden zu können, gibt es den `ObjectKind`-Typ:

```

data ObjectKind =
    FRAME
  | LABEL
  | MESSAGE
  | BUTTON
  ...

```

Für neue Widgets muß hier ein entsprechender Konstruktor hinzugefügt werden. Er steht im Modul `GUIObjectKind` in `kernel/GUIObjectKind.hs`.

- Die Widget-Methoden: Der `Methods`-Typ enthält eine Sammlung verschiedener Zugriffsmethoden auf Objekte. Da sich für unterschiedliche Objekte die Tcl-Syntax z.B. für das Setzen einer Konfigurationsoption oder für die Zerstörung des Objektes unterschiedlich sein kann, liefern diese Methoden eine einheitliche Schnittstelle für den Zugriff auf verschiedene Objekte (dazu mehr im nächsten Abschnitt). Für einfache Widgets reichen hier aber meist die Standard-Methoden in den `defMethods` im Modul `Core`. So wird man auch selten *alle* Methoden selbst schreiben, in der Regel reicht es, die `defMethods` als Ausgangspunkt zu nehmen und evtl. einzelne Methoden durch eigene zu überschreiben.

Will man keine der Default-Methoden überschreiben, reicht es, die Funktion

```
createWidget :: GUIOBJECT -> ObjectKind -> IO GUIOBJECT
```

aufzurufen. Diese ruft dann `createGUIObject` auf und setzt dabei die Default-Methoden.

Ein Beispiel für einen einfachen Widget-Konstruktor (Label):

```
newLabel :: Container par => par -> [Config (Label a)] -> IO (Label a)
newLabel par cnf =
  do
    w <- createWidget (toGUIObject par) LABEL -- GUIOBJECT erzeugen
    configure (Label w) cnf                  -- Konfigurationen setzen
```

`configure` aus dem Modul `Computation` wendet dabei einfach eine Liste von Konfigurationen auf ein Objekt an und hat die folgende Funktionalität:

```
configure :: w -> [Config w] -> IO w
```

5.2 Canvasitems

Für die Erzeugung von Canvas-Objekten gibt es eine ähnliche Funktion im Modul `CanvasItemAux`:

```
createCanvasItem :: CanvasItem w =>
  Canvas -> -- enthaltender Canvas
  CanvasItemKind -> -- Objekttyp
  (GUIOBJECT -> w) -> -- Wrap
  [Config w] -> -- Konfigurationsoptionen
  Coord -> -- initiale Position
  IO w
```

Diese Funktion gibt im Gegensatz zur oben erwähnten `createGUIObject` keinen `GUIOBJECT`-Wert, sondern das „echte“ Canvas-Objekt zurück (`createGUIObject` ist allgemeiner, da es auch von den Funktionen `createCanvasItem` und `createMenuItem`, s.u., benutzt wird). Da diese Funktion somit auch die externe Repräsentation des Objektes kennt, ist es hier auch möglich, die Liste der Konfigurationen mit zu übergeben. Diese werden dann automatisch angewendet. Parameter dieser Funktion sind:

- Der Canvas, der das Objekt enthalten soll.
- Ein Wert, der für Typ des Objektes steht (ähnlich wie `ObjectKind` für Widgets). Der Typ `CanvasItemKind` steht ebenfalls im Modul `GUIObjectKind` in `kernel/GUIObjectKind.hs`:

```
data CanvasItemKind =
    ARC
  | LINE
  ...
```

Für neue Objekttypen muß hier ebenfalls ein entsprechender Konstruktor hinzugefügt werden.

- Eine Funktion, die bei Eingabe des erzeugten `GUIOBJECT`-Wertes das zu erzeugende Canvas-Objekt liefert. Dies ist meist ein partiell parametrisierter Konstruktor des externen Typen des erzeugten Canvas-Objektes.
- Eine Liste von Konfigurationen für das zu erzeugende Canvas-Objekt.
- Die initiale Position auf dem Canvas.

Ein Beispiel (der Linien-Konstruktor aus dem Modul `Line`):

```
createLine :: Canvas -> [Config Line] -> IO Line
createLine cnv ol = createCanvasItem cnv LINE Line ol [(0,0),(0,0)]
```

5.3 Menüelemente

Zur Erzeugung von Menüelementen gibt es schließlich die folgende Funktion:

```
createMenuItem :: MenuItem w =>
    Menu -> -- Menü-Container
    MenuItemKind -> -- Objekttyp
    (GUIOBJECT -> w) -> -- Wrap
    [Config w] -> -- Konfigurationen
    IO w
```

Diese Funktion gibt ebenfalls das „echte“ Menüelement, bzw. dessen externe Repräsentation, zurück. Sie funktioniert im Prinzip genau wie `createCanvasItem`. Die Parameter sind:

- Der enthaltende Menü-Container (dieser kann einem `MenuButton`-, `MenuCascade`-, oder `Window`-Objekt zugeordnet werden).
- Ein Wert, der für den Typ des Objektes steht (wie `ObjectKind` für Widgets). Der Typ `MenuItemKind` steht auch im Modul `GUIObjectKind` in `kernel/GUIObjectKind.hs`:

```
data MenuItemKind =
    MENCASCADE
  | MENCMD
  ...
```


- Ein „Wrapper“, der bei Eingabe des erzeugten `GUIOBJECT`-Wertes die externe Repräsentation des Menüelementes liefert (siehe `CanvasItems`, dies ermöglicht wiederum das Verarbeiten der Konfigurationen innerhalb dieser Funktion).
- Die Liste der Konfigurationen.

6 Objekt-Methoden

6.1 Tcl-Syntax

Wie bereits erwähnt dienen die Objekt-Methoden einem einheitlichen Umgang mit verschiedenen Objekten bzw. deren unterschiedlicher Tcl-Syntax (bei Erzeugung, Packen usw.). Genauer gesagt erzeugen sie die jeweils erforderlichen Tcl-Kommandos in Form eines `TclScript`:

```
type TclCmd = String           -- ein einzelnes Kommando
type TclScript = [TclCmd]     -- eine Tcl-Befehlsfolge
```

6.2 Tcl-Skripte und Kommandos ausführen

In bestimmten Situationen, z.B. bei der Implementierung Objekt-spezifischer Konfigurationsoptionen, kann es sein, daß man solche Tcl-Skripte oder Kommandos selbst ausführen muß. Hierzu können die folgenden Funktionen aus dem Modul `Wish` benutzt werden:

```
execTclScript :: TclScript -> IO () -- Ausführen eines Tcl-Skriptes
execCmd       :: TclCmd    -> IO () -- Ausführen eines einzelnen Kommandos
```

(Kompliziertere Methoden, die auch die Antwort der `Wish` in Form einer `Tcl-Response` liefern, findet man dort ebenfalls. Auf diese möchte ich jedoch hier nicht weiter eingehen.)

6.3 Der Methods-Typ

Der `Methods`-Typ hat die folgenden Felder:

```
data Methods =
  Methods { -- Konfigurationsoption abfragen
    cgetCmd      :: ObjectName -> ConfigID -> TclScript,

    -- eine List von Konfigurationen setzen
    csetCmd      :: ObjectName -> [ConfigOption] -> TclScript,

    -- das Objekt erzeugen
    createCmd    :: ObjectName -> ObjectKind -> ObjectName ->
                  ObjectID -> [ConfigOption] -> TclScript,

    -- das Objekt mit Tk's "pack geometry manager" packen
    -- für "nicht-Widgets" leer
    packCmd      :: ObjectName -> [PackOption] -> TclScript,
```

```

-- das Objekt mit Tk's "grid geometry manager" packen
-- für "nicht-Widgets" leer
gridCmd      :: ObjectName -> [GridPackOption] -> TclScript,

-- das Objekt zerstören
destroyCmd   :: ObjectName -> TclScript,

-- Events an dieses Objekt binden
bindCmd      :: ObjectName -> BindTag -> [WishEvent] ->
              EventInfoSet -> TclScript,

-- Event-Bindung werfen
unbindCmd    :: ObjectName -> BindTag -> [WishEvent] ->
              TclScript,

-- zum aufräumen bei der Zerstörung, i.d.R. leer
cleanupCmd   :: ObjectID -> ObjectName -> TclScript }

```

Auf den Typ `ObjectName` möchte ich hier nicht weiter eingehen. Er wird durch die oben erwähnten `GUIOBJECT`-Konstruktoren automatisch erzeugt und enthält den Tcl-Pfad des entsprechenden Objektes. Es reicht zu wissen, daß er eine Instanz der Klasse `Show` ist und so auf den Tcl-Bezeichner des Objektes zugegriffen werden kann. Weiter wäre anzumerken, daß nicht für jedes Objekt tatsächlich *alle* obigen Parameter gebraucht werden. Dies ist vor allem Abhängig von der Art der Tk-Objekte (so braucht man z.B. das Vater-Objekte bei der Erzeugung nur zum Zugriff auf Text-Items).

- **cgetCmd:**

Erzeugt ein Tcl-Skript, daß den Wert einer Konfigurationsoption eines Objektes abfragt. Da der korrespondierende Wert immer eine Instanz der Klasse `GUIValue` ist, die eine `Read`-Instanz voraussetzt, kann er immer parsiert und zurückgegeben werden.

Beispiel (die Default-Methode):

```

tkGetWidgetConfig :: ObjectName -> -- Tcl-Pfad des Objektes
                  ConfigID ->    -- Tcl-Bezeichner der Konfiguration
                  TclScript
tkGetWidgetConfig name cid = [(show name) ++ " cget -" ++ cid]

```

- **csetCmd:**

Erzeugt ein Tcl-Skript, das eine Liste von Konfigurationen setzt. Der Typ `ConfigOption` ist einfach ein Tupel aus `ConfigID` (= `String`) und `GUIVALUE`.

Beispiel (die Default-Methode):

```

tkSetWidgetConfigs :: ObjectName -> -- Tcl-Pfad des Objektes
                   [ConfigOption] -> -- Liste der Konfigurationen
                   TclScript
tkSetWidgetConfigs _ [] = []

```

```
tkSetWidgetConfigs name args =
    [show name ++ " configure " ++ showConfigs args]
```

Die Funktion `showConfigs` kann zur bequemen Ausgabe einer Liste von Konfigurationen in den korrespondierenden `Tcl-String` verwendet werden:

```
showConfigs :: [(ConfigID, GUIVALUE)] -> String
```

- **createCmd:**

Erzeugt ein `Tcl-Skript`, das das korrespondierende `Tk-Objekt` konstruiert. Die dabei übergebene Liste von Konfigurationen kann benutzt werden, um intern bestimmte vom `Tk-Default` abweichende Konfigurationen für dieses Objekt vorzudefinieren. Die im `HTk-Konstruktor` übergebenen Konfigurationen werden hier nicht verarbeitet (dies geht aus Gründen, die ich hier nicht näher erläutern will auch gar nicht).

Beispiel (die Default-Methode):

```
tkCreateWidget :: ObjectName ->      -- Tcl-Pfad des Vaterobjektes
                        ObjectKind -> -- Objekttyp
                        ObjectName -> -- Tcl-Pfad des Objektes
                        ObjectID ->   -- ID des Objektes auf HTk-Seite
                        [ConfigOption] -> TclScript
tkCreateWidget _ kind name _ opts =
    [show kind ++ " " ++ show name ++ " " ++ showConfigs opts]
```

- **packCmd:**

Erzeugt ein `Tcl-Skript`, das ein bereits auf `Tk-Seite` existierendes *Widget* mit dem *pack geometry manager* packt. Diese Methode wird man niemals überschreiben, denn das Packen eines Widgets ist auf `Tk-Seite` einheitlich. Für nicht-Widgets sollte das erzeugte Skript leer sein.

Beispiel (die Default-Methode):

```
tkPack :: ObjectName ->      -- Tcl-Pfad des Objektes
                        [PackOption] -> -- Liste der Optionen für den geometry manager
                        TclScript
tkPack name opts = ["pack " ++ show name ++ " " ++ showPackOptions opts]
```

- **gridCmd:**

Erzeugt ein `Tcl-Skript`, das ein bereits auf `Tk-Seite` existierendes *Widget* mit dem *grid geometry manager* packt. Auch diese Methode wird man niemals überschreiben (das Packen eines Widgets ist auf `Tk-Seite` einheitlich). Für nicht-Widgets sollte das erzeugte Skript leer sein.

Beispiel (die Default-Methode):

```
tkGrid :: ObjectName ->      -- Tcl-Pfad des Objektes
                        [GridPackOption] -> -- Liste der Optionen für den geometry manager
                        TclScript
tkGrid name opts =
    ["grid " ++ show name ++ " " ++ showGridPackOptions opts]
```

- **destroyCmd:**

Erzeugt ein Tcl-Skript, das ein Tk-Objekt zerstört.

Beispiel (die Default-Methode):

```
tkDestroyWidget :: ObjectName ->    -- Tcl-Pfad des Objektes
                                TclScript
tkDestroyWidget name = ["destroy " ++ show name]
```

- **bindCmd:**

Erzeugt ein Tcl-Skript, das Events an ein Tk-Objekt bindet. Da auch diese Methode niemals überschrieben werden muß, will ich hier darauf nicht weiter eingehen. Wichtig ist nur, daß die Show-Instanz des `ObjectName` den Tcl-Pfad des Objektes liefert, aber dies ist in HTk sowieso Konvention bzw. wird durch die Methoden zur Erzeugung der internen `GUIOBJECT`-Repräsentation sichergestellt. Zu bemerken wäre höchstens noch, daß wir uns für HTk auf einen vereinfachten Umgang mit den von Tcl zurückzugebenen Event-Informationen (Position, Button, etc.) geeinigt haben, so daß hier immer dasselbe Event-Info-Set verwendet wird (auch dieser Typ muß daher hier nicht näher erläutert werden). Der `WishEvent`-Typ sollte von der Benutzerseite her bekannt sein.

Beispiel (die Default-Methode):

```
tkBindWidget :: ObjectName ->        -- Tcl-Pfad des Objektes
                                BindTag ->        -- historisch, rausnehmen !!
                                [WishEvent] ->      -- Liste der Events
                                EventInfoSet ->     -- Liste der zurückzugebenden
                                                -- Event-Informationen
                                TclScript
tkBindWidget nm bindTag wishEvents eventInfoSet =
  let doBind = "bind " ++ show nm ++ " " ++
    delimitString (foldr (\ event soFar -> showP event soFar)
      "" wishEvents) ++ " " ++
    mkBoundCmdArg bindTag eventInfoSet
  in [doBind]
```

- **unbindCmd:**

Erzeugt ein Tcl-Skript, daß Event-Bindungen (s.o.) rückgängig macht, d.h. auf Tcl-Seite für diese Events ein leeres Kommando bindet. Auch diese Methode wird niemals überschrieben werden müssen.

Beispiel (die Default-Methode):

```
tkUnbindWidget :: ObjectName ->
                                BindTag ->
                                [WishEvent] -> TclScript
tkUnbindWidget nm bindTag wishEvents =
  let doUnBind = "bind " ++ show nm ++ " " ++
    delimitString (foldr (\ event soFar -> showP event soFar)
      "" wishEvents) ++ " {}"
  in [doUnBind]
```

- **cleanupCmd:**

Dieses Kommando ist zur Freigabe auf Tcl-Seite benutzter Ressourcen bei der Zerstörung eines Objektes vorgesehen. Bisher wird es jedoch nicht benutzt. Die default-Methode ist daher leer.